Data-Mart-Erstellung in SQL (DLBDSPBDM01_D)

Erarbeitungs-/Reflexionsphase: Implementierung der Buchtausch-App-Datenbank

Miskovic, Dominik · Matrikelnummer: 14133980 · Kurs: DLBDSPBDM01_D

Entität: Benutzer

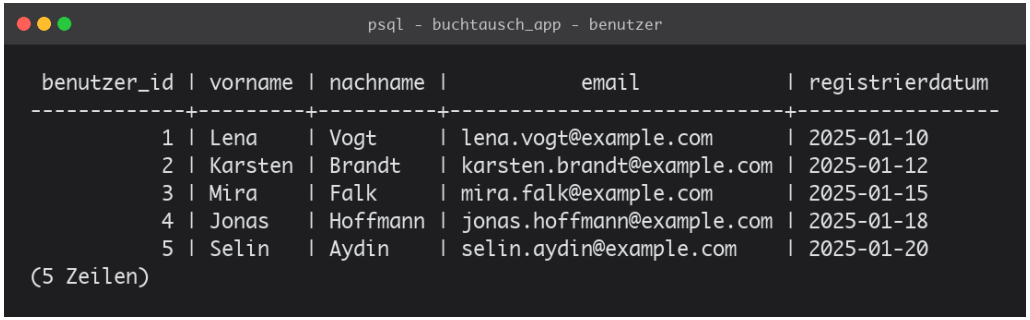
CREATE TABLE (ggf. gekürzt)

```
CREATE TABLE benutzer (
    benutzer_id      SERIAL PRIMARY KEY,
    vorname           VARCHAR(50)  NOT NULL,
    nachname          VARCHAR(50)  NOT NULL,
    email             VARCHAR(100) NOT NULL UNIQUE,
    telefon           VARCHAR(20),
    registrierdatum   DATE NOT NULL DEFAULT CURRENT_DATE
);
```

Testfall

```
SELECT benutzer_id, vorname, nachname, email, registrierdatum
FROM benutzer ORDER BY registrierdatum LIMIT 5;
```

email ist UNIQUE (Constraint aus dem Tutor-Feedback).



Entität: Adresse

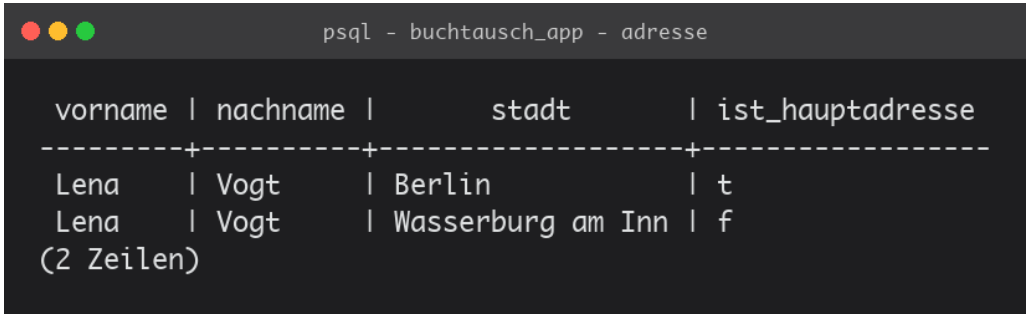
CREATE TABLE (ggf. gekürzt)

```
CREATE TABLE adresse (
    adresse_id     SERIAL PRIMARY KEY,
    benutzer_id     INTEGER NOT NULL REFERENCES benutzer(benutzer_id),
    strasse, hausnummer, plz, stadt ...
    breitengrad DECIMAL(9,6), laengengrad DECIMAL(9,6),
    ist_hauptadresse BOOLEAN NOT NULL DEFAULT FALSE
);
-- max. 1 Hauptadresse je Person:
CREATE UNIQUE INDEX ux_adresse_hauptadresse
    ON adresse (benutzer_id) WHERE ist_hauptadresse = TRUE;
```

Testfall

```
SELECT b.vorname, b.nachname, a.stadt, a.ist_hauptadresse
FROM adresse a JOIN benutzer b USING (benutzer_id)
WHERE b.benutzer_id = 1;
```

Nach Tutor-Feedback von 1:1 auf 1:N geändert: eine Person kann mehrere Adressen haben.



Entität: Verlag

CREATE TABLE (ggf. gekürzt)

```
CREATE TABLE verlag (
    verlag_id  SERIAL PRIMARY KEY,
    name        VARCHAR(100) NOT NULL,
    land        VARCHAR(50)
);
```

Testfall

```
SELECT verlag_id, name, land FROM verlag ORDER BY name LIMIT 5;
```

Lookup-Entität, vermeidet Wiederholung von Verlagsnamen bei jedem Buch.

psql - buchtausch_app - verlag

```
 verlag_id |         name          |    land
-----------+-----------------------+-------------
         9 | btb Verlag            | Deutschland
         5 | Diogenes Verlag       | Schweiz
         3 | dtv                   | Deutschland
         7 | Heyne Verlag          | Deutschland
        10 | Kiepenheuer & Witsch | Deutschland
(5 Zeilen)
```

Entität: Genre

CREATE TABLE (ggf. gekürzt)

```
CREATE TABLE genre (
    genre_id     SERIAL PRIMARY KEY,
    bezeichnung   VARCHAR(50) NOT NULL UNIQUE
);
```

Testfall

```
SELECT genre_id, bezeichnung FROM genre ORDER BY genre_id;
```

10 Genres als Dummy-Daten, UNIQUE verhindert Duplikate.



Entität: Sprache

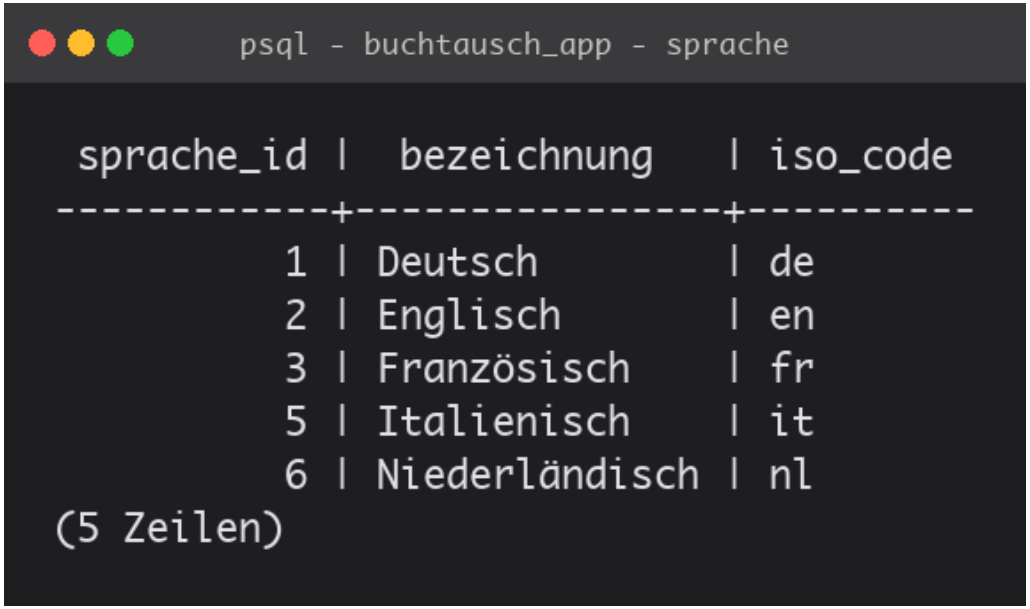
CREATE TABLE (ggf. gekürzt)

```
CREATE TABLE sprache (
    sprache_id   SERIAL PRIMARY KEY,
    bezeichnung   VARCHAR(50) NOT NULL,
    iso_code      CHAR(2) NOT NULL UNIQUE
);
```

Testfall

```
SELECT sprache_id, bezeichnung, iso_code
FROM sprache ORDER BY bezeichnung LIMIT 5;
```

ISO-639-1-Code als zusätzliches, eindeutiges Attribut.



Entität: Buch

CREATE TABLE (ggf. gekürzt)

```
CREATE TABLE buch (
    buch_id  SERIAL PRIMARY KEY,
    titel VARCHAR(200) NOT NULL, isbn VARCHAR(20) UNIQUE,
    erscheinungsjahr INTEGER CHECK (erscheinungsjahr BETWEEN 1450 AND ...),
    genre_id INTEGER REFERENCES genre(genre_id),
    verlag_id INTEGER REFERENCES verlag(verlag_id),
    sprache_id INTEGER NOT NULL REFERENCES sprache(sprache_id)
);
```

Testfall

```
SELECT bu.titel, bu.erscheinungsjahr, g.bezeichnung genre,
       v.name verlag, s.bezeichnung sprache
FROM buch bu JOIN genre g USING (genre_id)
JOIN verlag v USING (verlag_id) JOIN sprache s USING (sprache_id)
ORDER BY bu.titel LIMIT 5;
```

Bibliografische Ausgabe, bewusst getrennt vom Exemplar (siehe Phase 1).



Entität: Autor

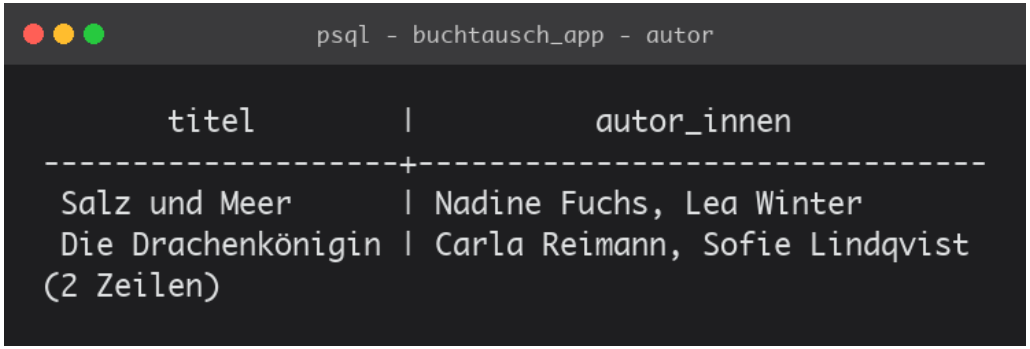
CREATE TABLE (ggf. gekürzt)

```
CREATE TABLE autor (
    autor_id   SERIAL PRIMARY KEY,
    vorname     VARCHAR(50) NOT NULL,
    nachname    VARCHAR(50) NOT NULL
);
CREATE TABLE buch_autor (   -- Aufloesung der N:M-Beziehung
    buch_id INTEGER REFERENCES buch(buch_id),
    autor_id INTEGER REFERENCES autor(autor_id),
    PRIMARY KEY (buch_id, autor_id)
);
```

Testfall

```
SELECT bu.titel, string_agg(a.vorname||' '||a.nachname, ', ') autor_innen
FROM buch bu JOIN buch_autor ba USING (buch_id)
JOIN autor a USING (autor_id)
GROUP BY bu.buch_id, bu.titel HAVING count(*) > 1;
```

Testfall zeigt gezielt Bücher mit mehreren Autor:innen (N:M-Fall).



Entität: Buch_Autor (Verbindungstabelle)

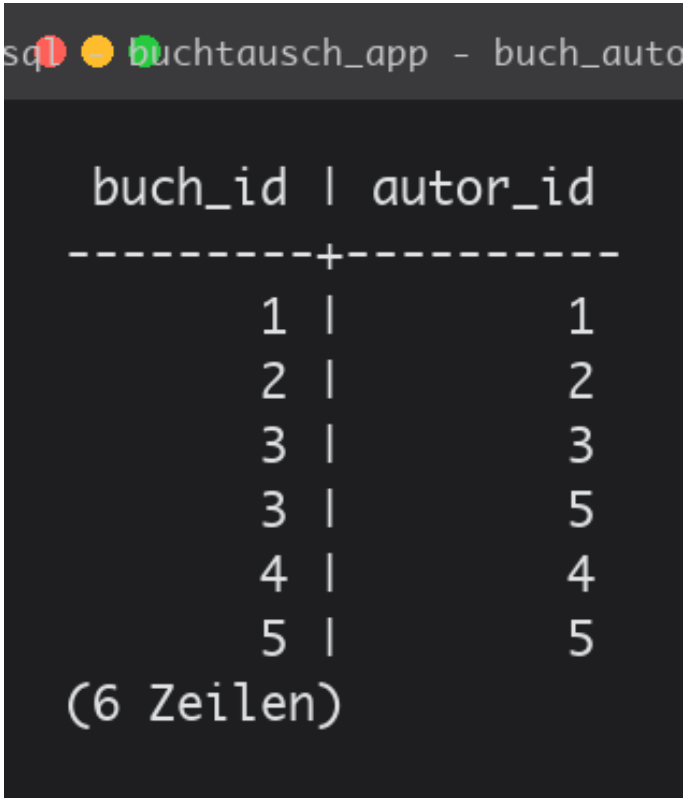
CREATE TABLE (ggf. gekürzt)

```
CREATE TABLE buch_autor (
    buch_id   INTEGER NOT NULL REFERENCES buch(buch_id),
    autor_id  INTEGER NOT NULL REFERENCES autor(autor_id),
    PRIMARY KEY (buch_id, autor_id)
);
```

Testfall

```
SELECT buch_id, autor_id FROM buch_autor
ORDER BY buch_id LIMIT 6;
```

Physische Umsetzung der N:M-Beziehung Buch-Autor als eigene Tabelle.



Entität: Exemplar

CREATE TABLE (ggf. gekürzt)

```
CREATE TABLE exemplar (
    exemplar_id  SERIAL PRIMARY KEY,
    buch_id INTEGER NOT NULL REFERENCES buch(buch_id),
    besitzer_id INTEGER NOT NULL REFERENCES benutzer(benutzer_id),
    zustand VARCHAR(20) NOT NULL CHECK (zustand IN
        ('neu','sehr gut','gut','gebraucht','stark gebraucht')),
    versand_moeglich BOOLEAN NOT NULL DEFAULT FALSE,
    max_leihdauer_tage INTEGER NOT NULL CHECK (max_leihdauer_tage > 0),
    eingestellt_am DATE NOT NULL DEFAULT CURRENT_DATE
);
```

Testfall

```
SELECT e.exemplar_id, bu.titel, be.vorname||' '||be.nachname besitzer_in,
       e.zustand, e.max_leihdauer_tage
FROM exemplar e JOIN buch bu USING (buch_id)
JOIN benutzer be ON be.benutzer_id = e.besitzer_id
ORDER BY e.exemplar_id LIMIT 5;
```

max_leihdauer_tage > 0 per CHECK erzwungen (Tutor-Feedback Punkt 3).



Entität: Abholort

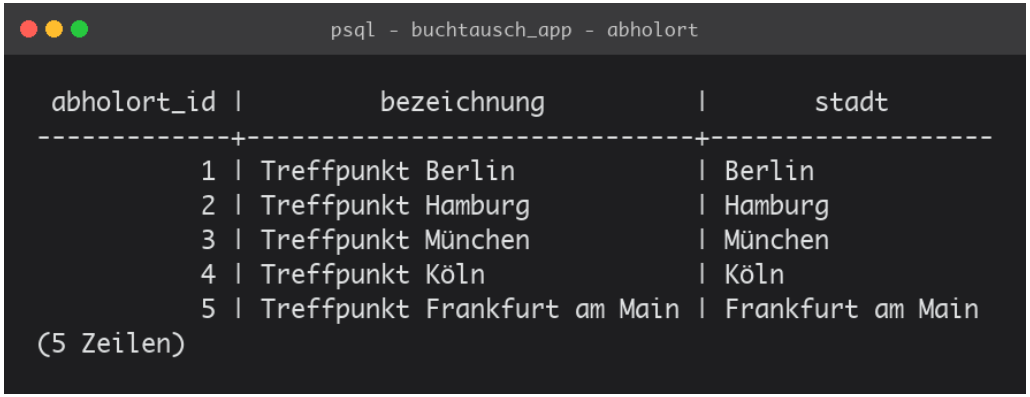
CREATE TABLE (ggf. gekürzt)

```
CREATE TABLE abholort (
    abholort_id  SERIAL PRIMARY KEY,
    benutzer_id   INTEGER NOT NULL REFERENCES benutzer(benutzer_id),
    bezeichnung, strasse, plz, stadt VARCHAR ...,
    breitengrad DECIMAL(9,6) NOT NULL,
    laengengrad DECIMAL(9,6) NOT NULL
);
```

Testfall

```
SELECT abholort_id, bezeichnung, stadt
FROM abholort ORDER BY abholort_id LIMIT 5;
```

Öffentliche Übergabepunkte, getrennt von der privaten Adresse.



Entität: Verfügbarkeitszeitfenster

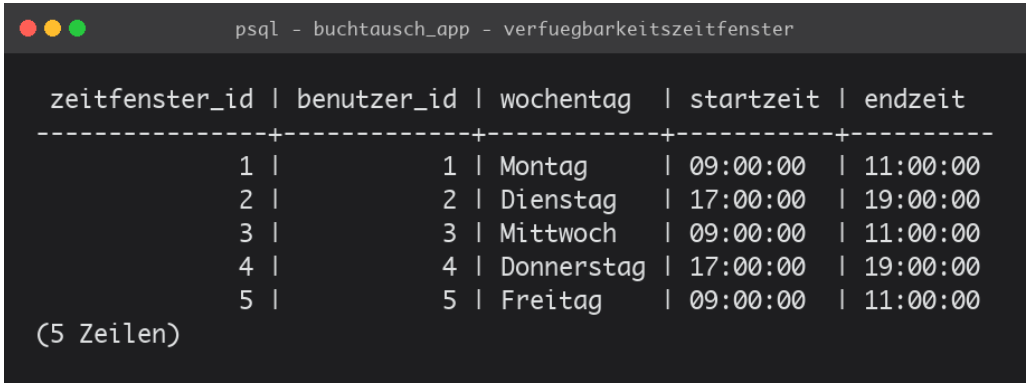
CREATE TABLE (ggf. gekürzt)

```
CREATE TABLE verfuegbarkeitszeitfenster (
    zeitfenster_id  SERIAL PRIMARY KEY,
    benutzer_id      INTEGER NOT NULL REFERENCES benutzer(benutzer_id),
    wochentag        VARCHAR(10) NOT NULL CHECK (wochentag IN
                      ('Montag', ..., 'Sonntag')),
    startzeit TIME NOT NULL, endzeit TIME NOT NULL,
    CONSTRAINT chk_zeitfenster_reihenfolge CHECK (endzeit > startzeit)
);
```

Testfall

```
SELECT zeitfenster_id, benutzer_id, wochentag, startzeit, endzeit
FROM verfuegbarkeitszeitfenster ORDER BY zeitfenster_id LIMIT 5;
```

endzeit > startzeit per CHECK validiert (Tutor-Feedback Punkt 3).



Entität: Ausleihvorgang

CREATE TABLE (ggf. gekürzt)

```
CREATE TABLE ausleihvorgang (   -- reifizierte Ternärbeziehung AUSLEIHT
    ausleihvorgang_id  SERIAL PRIMARY KEY,
    exemplar_id INTEGER NOT NULL REFERENCES exemplar(exemplar_id),
    entleiher_id INTEGER NOT NULL REFERENCES benutzer(benutzer_id),
    zeitfenster_id INTEGER NOT NULL REFERENCES verfuegbarkeitszeitfenster(...),
    ausleihdatum DATE NOT NULL DEFAULT CURRENT_DATE,
    geplantes_rueckgabedatum DATE NOT NULL,
    tatsaechliches_rueckgabedatum DATE,
    status VARCHAR(20) NOT NULL DEFAULT 'aktiv'
        CHECK (status IN ('aktiv','zurueckgegeben','ueberfaellig'))
);
-- Trigger: entleiher_id darf nicht besitzer_id des Exemplars sein.
```

Testfall

```
SELECT av.ausleihvorgang_id, bu.titel,
       en.vorname||' '||en.nachname entleiher_in,
       av.ausleihdatum, av.status
FROM ausleihvorgang av JOIN exemplar e USING (exemplar_id)
JOIN buch bu USING (buch_id)
JOIN benutzer en ON en.benutzer_id = av.entleiher_id
ORDER BY av.ausleihvorgang_id LIMIT 6;
```

Reifizierung der Ternärbeziehung AUSLEIHT aus Phase 1 (eigener PK nötig, damit Bewertung darauf verweisen kann).



Entität: Angebot

CREATE TABLE (ggf. gekürzt)

```
CREATE TABLE angebot (   -- reifizierte Ternärbeziehung ANGEBOTEN_ALS
    angebot_id   SERIAL PRIMARY KEY,
    exemplar_id   INTEGER NOT NULL REFERENCES exemplar(exemplar_id),
    abholort_id   INTEGER NOT NULL REFERENCES abholort(abholort_id),
    zeitfenster_id INTEGER NOT NULL REFERENCES verfuegbarkeitszeitfenster(...),
    aktiv         BOOLEAN NOT NULL DEFAULT TRUE,
    UNIQUE (exemplar_id, abholort_id, zeitfenster_id)
);
```

Testfall

```
SELECT an.angebot_id, bu.titel, ao.stadt, an.aktiv
FROM angebot an JOIN exemplar e USING (exemplar_id)
JOIN buch bu USING (buch_id)
JOIN abholort ao USING (abholort_id)
ORDER BY an.angebot_id LIMIT 6;
```

Reifizierung der zweiten Ternärbeziehung aus Phase 1.

psql - buchtausch_app - angebot

angebot_id	titel	stadt	aktiv
1	Der Winterpfad	Berlin	t
2	Mord im Nebel	Hamburg	t
3	Die Drachenkönigin	München	t
4	Wege durch die Stille	Köln	t
5	Der letzte Leuchtturm	Frankfurt am Main	t
6	Sternenstaub	Stuttgart	t

(6 Zeilen)

Entität: Bewertung

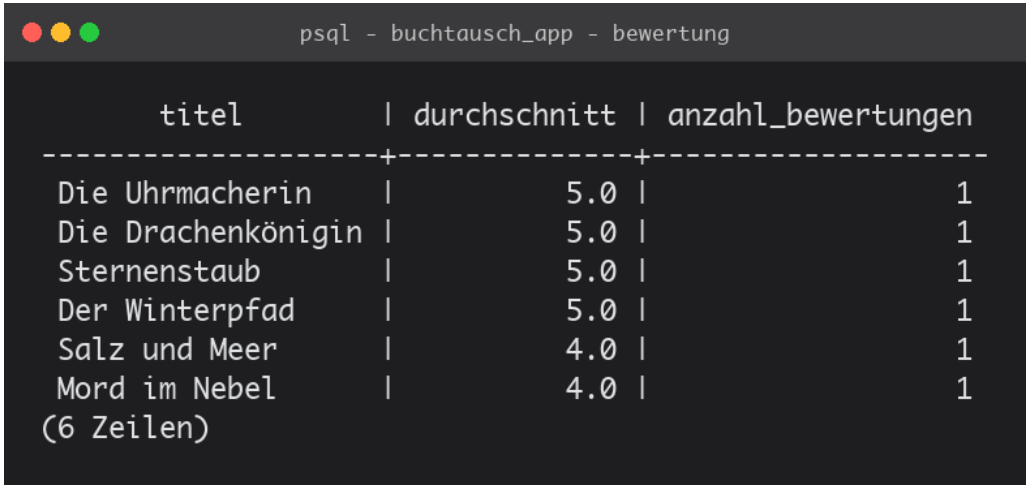
CREATE TABLE (ggf. gekürzt)

```
CREATE TABLE bewertung (
    bewertung_id       SERIAL PRIMARY KEY,
    ausleihvorgang_id   INTEGER NOT NULL UNIQUE
                        REFERENCES ausleihvorgang(ausleihvorgang_id),
    sternebewertung     INTEGER NOT NULL CHECK (sternebewertung BETWEEN 1 AND 5),
    kommentar           TEXT,
    erstellt_am         DATE NOT NULL DEFAULT CURRENT_DATE
);
-- Trigger: nur INSERT, wenn ausleihvorgang.status = 'zurueckgegeben'.
```

Testfall

```
SELECT bu.titel, ROUND(AVG(bw.sternebewertung),1) durchschnitt,
       count(*) anzahl_bewertungen
FROM bewertung bw JOIN ausleihvorgang av USING (ausleihvorgang_id)
JOIN exemplar e USING (exemplar_id) JOIN buch bu USING (buch_id)
GROUP BY bu.buch_id, bu.titel ORDER BY durchschnitt DESC LIMIT 6;
```

Feedback Phase 1 umgesetzt: Bewertung hängt jetzt am Ausleihvorgang, nicht direkt an Buch/Benutzer.



Bonus: Räumliche Suche ("Bücher in meiner Nähe")

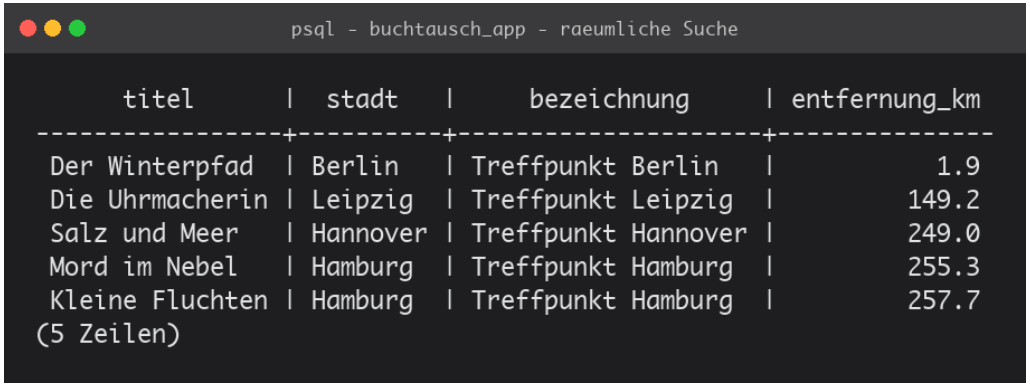
SQL

```
-- Umkreissuche per Haversine-Formel (ohne PostGIS-Erweiterung),
-- Referenzpunkt hier beispielhaft: Berlin (52.520 / 13.405)
SELECT ..., ROUND((6371 * acos(
    cos(radians(:lat)) * cos(radians(ao.breitengrad)) *
    cos(radians(ao.laengengrad) - radians(:lng)) +
    sin(radians(:lat)) * sin(radians(ao.breitengrad))
))::numeric, 1) AS entfernung_km
FROM angebot a JOIN exemplar e USING (exemplar_id)
JOIN buch bu USING (buch_id) JOIN abholort ao USING (abholort_id)
WHERE a.aktiv = TRUE ORDER BY entfernung_km;
```

Testfall

```
-- siehe CREATE-Statement oben, direkt ausführbar mit festen Koordinaten:
SELECT bu.titel, ao.stadt, ao.bezeichnung, entfernung_km ...
ORDER BY entfernung_km LIMIT 5;
```

Setzt die Anforderung "Bücher in meiner Nähe auf einer Karte anzeigen" aus der Anforderungsspezifikation um.



Bonus: Negativtest 1 (Datenintegrität)

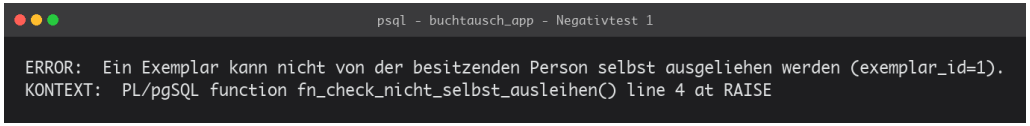
SQL

```
CREATE OR REPLACE FUNCTION fn_check_nicht_selbst_ausleihen()
RETURNS TRIGGER AS $$
BEGIN
  IF NEW.entleiher_id = (SELECT besitzer_id FROM exemplar
                         WHERE exemplar_id = NEW.exemplar_id) THEN
    RAISE EXCEPTION 'Exemplar kann nicht von der besitzenden Person
                     selbst ausgeliehen werden (exemplar_id=%).',
                     NEW.exemplar_id;
  END IF;
  RETURN NEW;
END; $$ LANGUAGE plpgsql;
```

Testfall

```
-- Testfall: Besitzer:in von Exemplar 1 versucht, es selbst auszuleihen
INSERT INTO ausleihvorgang (exemplar_id, entleiher_id, zeitfenster_id,
    ausleihdatum, geplantes_rueckgabedatum)
VALUES (1, (SELECT besitzer_id FROM exemplar WHERE exemplar_id=1),
    1, CURRENT_DATE, CURRENT_DATE + 7);
```

Erwartetes und tatsächliches Ergebnis: INSERT wird vom Trigger abgelehnt.



Bonus: Negativtest 2 (Tutor-Feedback Punkt 1)

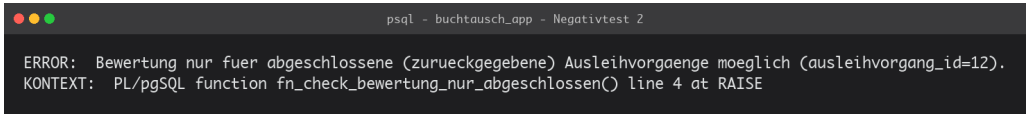
SQL

```
CREATE OR REPLACE FUNCTION fn_check_bewertung_nur_abgeschlossen()
RETURNS TRIGGER AS $$
BEGIN
  IF (SELECT status FROM ausleihvorgang
      WHERE ausleihvorgang_id = NEW.ausleihvorgang_id) <> 'zurueckgegeben' THEN
    RAISE EXCEPTION 'Bewertung nur fuer abgeschlossene Ausleihvorgaenge
                     moeglich (ausleihvorgang_id=%).', NEW.ausleihvorgang_id;
  END IF;
  RETURN NEW;
END; $$ LANGUAGE plpgsql;
```

Testfall

```
-- Testfall: Bewertung fuer einen noch aktiven Ausleihvorgang (id=12)
INSERT INTO bewertung (ausleihvorgang_id, sternebewertung, kommentar)
VALUES (12, 5, 'Sollte scheitern, da Ausleihvorgang 12 noch aktiv ist');
```

Setzt Tutor-Feedback direkt technisch um: Bewertung nur nach abgeschlossener Ausleihe.



Kurze Zusammenfassung der Implementierung

Das Datenbankschema der Buchtausch-App wurde in PostgreSQL 16 umgesetzt und besteht aus 14 Tabellen: den 13 Entitäten aus dem ER-Modell der Konzeptionsphase sowie der Verbindungstabelle buch_autor zur Auflösung der N:M-Beziehung zwischen Buch und Autor:in. Beide Ternärbeziehungen aus Phase 1 (AUSLEIHT und ANGEBOTEN_ALS) wurden als eigene Tabellen mit Surrogatschlüssel umgesetzt (ausleihvorgang, angebot). Das ist eine Standardtechnik, wenn eine Beziehung selbst wieder referenziert werden muss, wie es das Tutor-Feedback zur Bewertung erforderte.

Alle drei Rückmeldungen aus dem Feedback zu Phase 1 wurden eingearbeitet: Die Bewertung ist jetzt über einen UNIQUE-Fremdschlüssel an einen konkreten Ausleihvorgang gekoppelt statt direkt an Buch und Benutzer:in, zusätzlich per Trigger abgesichert, dass nur abgeschlossene ("zurueckgegeben") Ausleihvorgänge bewertet werden können. Die Beziehung zwischen Benutzer und Adresse wurde von 1:1 auf 1:N geändert, sodass mehrere Adressen je Person möglich sind. Und die geforderten Constraints wurden konsequent implementiert: CHECK-Constraints auf Sternebewertung (1–5), auf positive Leihdauer und auf die zeitliche Reihenfolge von Ausleih- und Zeitfenster-Daten, UNIQUE auf die E-Mail-Adresse sowie ein zusätzlicher Trigger, der verhindert, dass Nutzende ihr eigenes Exemplar ausleihen.

Die Datenbank wurde mit insgesamt über 175 Dummy-Datensätzen befüllt (mindestens 10 pro Tabelle), alle referenziell konsistent und alle Geschäftsregeln respektierend. Die räumliche Suchanforderung ("Bücher in meiner Nähe") wurde über eine Haversine-Distanzformel direkt in SQL umgesetzt, ganz ohne zusätzliche Erweiterung wie PostGIS, und liefert nachweislich plausible, nach Entfernung sortierte Ergebnisse. Zwei Negativtests belegen, dass die implementierten Trigger tatsächlich greifen und fehlerhafte Eingaben zuverlässig zurückweisen.